QALight

Frontend-розробка

Заняття 20. Форми

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний

!

Форми
Властивості та методи форми. Фокусування. Події та методи. Валідація форм.
Створення модального вікна з формою регістрації, та валідація полів форми.

Форми

Властивості та методи форми. Фокусування. Події та методи.

Основні події та методи форм

Теорія: forms

Основні події та методи форм у JavaScript дозволяють керувати відправкою даних, відстежувати зміни полів та взаємодіяти з елементами інтерфейсу.

Основні події форм:

- **submit** - виникає під час відправки форми (наприклад, при натисканні на кнопку типу submit або клавішу Enter). Найчастіше використовується для **валідації** даних та скасування стандартного перезавантаження сторінки за допомогою event.preventDefault().
- **input** - спрацьовує щоразу при зміні значення в полі введення (<input>, <textarea>, <select>), наприклад, при введенні кожного нового символу.
- **change** - виникає, коли зміна значення елемента завершена. Для текстових полів це відбувається при втраті фокусу (**blur**), а для прапорців (**checkbox**), перемикачів (**radio**) та випадаючих списків (<**select**>) — одразу при їх виборі.
- **focus** та **blur** - виникають, коли елемент отримує або втрачає фокус.

Основні методи форм:

- form.submit() — дозволяє програмно ініціювати та запустити відправку форми на сервер без необхідності натискати на кнопку.
- form.reset() — очищає всі поля форми, повертаючи їх до початкових значень.

Модальне вікно

Теорія: forms

Модальне вікно js - це спливаючий елемент користувацького інтерфейсу, який блокує весь інший контент сторінки, щоб зосередити увагу користувача на конкретному завданні або інформації. Воно зазвичай містить важливе повідомлення, форму для заповнення даних, попередження або підтвердження операції.

Приклади використання модальних вікон охоплюють запити підтвердження дій (наприклад, підтвердження видалення елемента), форми реєстрації або входу на сайт, сповіщення про акції або знижки, сповіщення про помилку та багато іншого. Модальні вікна можуть бути як простими, що містять тільки текст і кнопки, так і більш складними, з інтерактивними елементами, відео або кастомним контентом.

Переваги використання модальних вікон включають підвищення зручності користувачів, оскільки вони пропонують швидкий доступ до важливої інформації без необхідності переходити на іншу сторінку. Вони також допомагають сфокусувати увагу користувача на конкретному завданні або дії, що може поліпшити конверсію і залученість. Крім того, модальні вікна можуть бути легко стилізовані та налаштовані відповідно до дизайну сайту, що дає змогу створювати єдиний і структурований користувацький досвід.

HTML5-елемент <dialog>

Теорія: forms

HTML-елемент **<dialog>** створює діалогове вікно або інший інтерактивний компонент (наприклад, спливаюче повідомлення чи форму)

Елемент <dialog> автоматично утримує фокус клавіатури всередині модального вікна та закривається натисканням клавіші Esc.

Основні можливості та атрибути:

- Атрибут **open**: вказує, що діалогове вікно є активним і відображається на екрані. Проте керувати його появою зазвичай радять через JavaScript.
- Псевдоелемент **::backdrop**: дозволяє легко стилізувати затемнений чи розмитий фон позаду модального вікна.
- Атрибут **method="dialog"**: якщо всередині тегу <dialog> розмістити форму з таким методом, її **сабміт автоматично закриє вікно**.

Режими роботи dialog

Теорія: forms

- Звичайний (немодальний): Вікно з'являється на сторінці, але користувач може далі взаємодіяти з рештою контенту поза ним.
- Модальний (**showModal**()): Вікно блокує взаємодію з рештою сторінки, додаючи навколо темний фон (підтримує псевдоклас ::backdrop).

Головні методи та властивості

- **show()** - відкриває діалогове вікно як звичайне.
- **showModal**() - відкриває вікно в модальному режимі.
- **close**() - закриває діалогове вікно (може приймати необов'язковий рядок returnValue).
- Атрибут **open** - якщо присутній в HTML, одразу показує вікно.

```
<dialog id="myDialog">
 <p>Привіт! Це діалогове вікно.</p>
 <form method="dialog">
   <button>Закрити</button>
 </form>
</dialog>
<button onclick="document.getElementById('myDialog').showModal()">Відкрити</button
```

Форми

Створення модального вікна з формою регістрації

Створення модального вікна

Практика: forms

```
<!-- Кнопка для виклику вікна -->
<button id="openModalBtn">Зареєструватися</button>

<!-- Модальне вікно →

<dialog id="regModal">

    <div class="modal-header">
        <h2>Реєстрація</h2>
        <button id="closeModalBtn" class="close-btn">&times;</button>
    </div>

  </dialog>
```

Форма реєстрації

Практика: forms

```
<form id="regForm">
    <div class="form-group">
        <label for="username">Ім'я користувача:</label>
        <input type="text" id="username" name="username" required minlength="3">
    </div>

    <div class="form-group">
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
    </div>

    <div class="form-group">
        <label for="password">Пароль:</label>
        <input type="password" id="password" name="password" required minlength="6">
    </div>

    <button type="submit" class="submit-btn">Створити акаунт</button>
</form>
```

Форма реєстрації (стилі)

Практика: forms

```
/* Стилізація фону модального вікна (оверлей) */
    dialog::backdrop {
        background-color: rgba(0, 0, 0, 0.5);
        backdrop-filter: blur(4px);
    }

    /* Контейнер діалогового вікна */
    dialog {
        border: none;
        border-radius: 8px;
        padding: 24px;
        width: 100%;
        max-width: 400px;
        box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
    }
```

Форма реєстрації (стилі)

Практика: forms

```css
/* Шапка модального вікна */
.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}
.modal-header h2 {
    margin: 0;
    font-size: 1.5rem;
}
.close-btn {
    background: none;
    border: none;
    font-size: 2rem;
    cursor: pointer;
    line-height: 1;
}
```

Форма реєстрації (стилі)

Практика: forms

```css
/* Елементи форми */
.form-group {
    margin-bottom: 15px;
}

.form-group label {
    display: block;
    margin-bottom: 5px;
    font-weight: 500;
}

.form-group input {
    width: 100%;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
}
```

Форма реєстрації (стилі)

Практика: forms

```css
/* Кнопка відправки */
.submit-btn {
    width: 100%;
    padding: 12px;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 4px;
    font-size: 1rem;
    cursor: pointer;
}

.submit-btn:hover {
    background-color: #0056b3;
}
```

Посилання на DOM-елементи

Практика: forms

```
// Отримуємо посилання на необхідні DOM-елементи

const modal = document.getElementById('regModal'); const openBtn =
document.getElementById('openModalBtn');

const closeBtn = document.getElementById('closeModalBtn');

const form = document.getElementById('regForm');
```

Відкриття модального вікна

Практика: forms

```
\
// Відкриття модального вікна за допомогою вбудованого методу showModal()
openBtn.addEventListener('click', () => {
    modal.showModal();
});
// Закриття вікна при натисканні на кнопку "хрестик"
closeBtn.addEventListener('click', () => {
    modal.close();
});
// Закриття вікна при кліку на область поза формою (на ::backdrop)
modal.addEventListener('click', (event) => {
    if (event.target === modal) {
        modal.close();
    }
});
```

Обробка відправки форми реєстрації

Практика: forms

```
// Обробка відправки форми реєстрації
form.addEventListener('submit', (event) => {
    event.preventDefault(); // Запобігаємо стандартному перезавантаженню сторінки

    // Отримуємо дані з полів форми
    const formData = new FormData(form);
    const data = Object.fromEntries(formData.entries());

    console.log('Дані реєстрації надіслано:', data);

    // Імітація успішного запиту: очищуємо форму та закриваємо модалку
    alert(`Користувач ${data.username} успішно зареєстрований!`);
    form.reset();
    modal.close();
});
```

Фокусування

Практика: forms

Для покращення взаємодії з користувачем (UX) важливо правильно налаштувати фокусування (Focus Management) всередині модального вікна.
\
Вбудований елемент <dialog> уже автоматично переносить фокус на перше поле форми при відкритті.

Однак, щоб форма працювала професійно та була зручною для керування з клавіатури, варто додати три важливі покращення:

1. **Автофокус на першому полі** після відкриття (через атрибут autofocus).
2. **Повернення фокусу на кнопку**, яка відкрила модалку, після її закриття.
3. **Візуальний фокус (Focus Ring)** для кращої навігації клавіатурою (Tab).

Додаємо autofocus

Практика: forms

Достатньо додати атрибут **autofocus** до першого інпуту. Браузер сам поставить туди курсор у момент виклику **showModal**().

```
<form id="regForm">


    <div class="form-group">
        <label for="username">Ім'я користувача:</label>
        <!-- Додано атрибут autofocus -->
        <input type="text" id="username" name="username" required minlength="3" autofocus>
    </div>
```

Підсвічування фокусу

Практика: forms

Замість стандартної синьої рамки браузера, зробимо м'яке та помітне підсвічування активного поля.

```
/* Стиль для полів та кнопок, коли вони в фокусі */
    .form-group input:focus,
    .submit-btn:focus,
    .close-btn:focus,
    .toggle-password:focus {
        outline: 2px solid #007bff;
        outline-offset: 2px;
        border-color: transparent;
    }
    /* Якщо користувач клікає мишкою, outline можна приховати (опціонально) */
    .form-group input:focus:not(:focus-visible),
    .submit-btn:focus:not(:focus-visible) {
        outline: none;
        border-color: #007bff;
        /* залишаємо тільки кольоровий бордер для мишки */
    }
```

Повернення фокусу

Практика: forms

Елемент <dialog> за замовчуванням намагається повернути фокус туди, звідки прийшов користувач.

Але якщо ви закриваєте модалку вручну через код (наприклад, після успішного fetch), варто зафіксувати елемент, який її відкрив

- Коли користувач відкриває форму за допомогою клавіші Enter, його фокус одразу опиняється в першому полі Ім'я користувача — він може відразу друкувати.
- Переміщення за допомогою кнопки Tab циклічно ходить лише всередині форми (це вбудована магія тегу <dialog>).
- Після закриття вікна (через Esc чи кнопку) фокус повертається точно на кнопку «Зареєструватися», і користувач може продовжити гортати сайт.

Відкриття модального вікна

Практика: forms

```
// Змінна для зберігання кнопки, яка викликала модальне вікно
let lastActiveElement;


:// Відкриття модального вікна

openBtn.addEventListener('click', () => {
    lastActiveElement = document.activeElement; // Запам'ятовуємо кнопку "Зареєструватися"
    modal.showModal();
    // Фокус автоматично впаде на інпут з атрибутом autofocus
});
```

Функція для закриття з поверненням фокусу

Практика: forms

```
// Функція для закриття з поверненням фокусу
function closeModalWindow() {
    modal.close();
    // Повертаємо фокус назад на кнопку, щоб користувач клавіатури не загубився на сторінці
    if (lastActiveElement) {
        lastActiveElement.focus();
    }
}
// Замінюємо modal.close() у всіх обробниках на нашу функцію:
closeBtn.addEventListener('click', closeModalWindow);

modal.addEventListener('click', (event) => {
    if (event.target === modal) {
        closeModalWindow();
    }
});
```

Обробка відправки форми реєстрації

Практика: forms

```
// Обробка відправки форми реєстрації
/ В обробнику submit (всередині блоку успішної відповіді fetch):// замість modal.close() викликаємо closeModalWindow();
    form.addEventListener('submit', (event) => {
        event.preventDefault(); // Запобігаємо стандартному перезавантаженню сторінки

        // Отримуємо дані з полів форми
        const formData = new FormData(form);
        const data = Object.fromEntries(formData.entries());
        console.log('Дані реєстрації надіслано:', data);

        // Імітація успішного запиту: очищуємо форму та закриваємо модалку
        alert(`Користувач ${data.username} успішно зареєстрований!`);
        form.reset();
        // modal.close();
        closeModalWindow()
    });
```

Форми

Валідація форм.

Основні підходи до валідації

Практика: forms

Валідація форм на JavaScript - це перевірка введених користувачем даних перед відправкою на сервер для запобігання помилкам та захисту від некоректного введення.

Основні підходи до валідації

- Жива валідація (input): Перевірка тексту безпосередньо під час введення. Дозволяє підсвічувати поле зеленим або червоним кольором у реальному часі.
- Валідація при втраті фокусу (blur): Перевірка поля, коли користувач переходить до наступного елемента. Це не дратує передчасними помилками під час введення.
- Вбудована HTML5 валідація (Constraint Validation API): JavaScript дозволяє взаємодіяти з атрибутами required, pattern, minlength. За допомогою методу input.checkValidity() можна дізнатися, чи відповідає поле правилам HTML5, а через input.setCustomValidity('текст') - задати власну помилку.

HTML5 Constraint Validation API

Практика: forms

HTML5 Constraint Validation API - це вбудований набір методів і властивостей JavaScript, що дозволяє розробникам взаємодіяти з механізмом перевірки форм, вбудованим у браузер. Цей API слугує сполучною ланкою між суто декларативними атрибутами перевірки HTML5 (такими як `required`, `pattern` і `type="email"`) та повністю кастомними алгоритмами валідації.

HTML5 Constraint Validation API (Основні методи)

Практика: forms

Основні методи. API додає кілька методів безпосередньо як до окремих елементів керування формою (наприклад, `<input>`, `<select>`, `<textarea>`), так і до самого елемента `<form>`:

checkValidity(): Перевіряє елемент або форму на відповідність заданим обмеженням валідації. Метод повертає `true`, якщо дані валідні, або `false`, якщо порушено будь-які обмеження. Він генерує подію `invalid` для елементів, що не пройшли перевірку, але не відображає помилки візуально для користувача.

reportValidity(): Працює так само, як і `checkValidity()`, але якщо елемент не проходить перевірку, метод також викликає стандартне спливаюче вікно браузера з повідомленням про помилку, щоб негайно поінформувати користувача.

setCustomValidity(message): Встановлює власне повідомлення про помилку для елемента введення. Якщо передати непорожній рядок, браузер автоматично позначає поле як невалідне та встановлює цей рядок, як повідомлення про помилку валідації.

Якщо передати порожній рядок (`""`), власна помилка видаляється, а поле знову позначається як валідне (за умови, що не порушено жодних інших стандартних обмежень).

HTML5 Constraint Validation API (Ключові властивості)

Практика: forms

Елементи форми отримують спеціальні властивості для програмного зчитування їхнього поточного стану перевірки (валідації):

willValidate: логічне значення, що вказує, чи підлягає елемент перевірці на відповідність обмеженням (наприклад, повертає `false`, якщо поле введення вимкнене або має атрибут `readonly`).

validationMessage: локалізований рядок, що містить конкретне повідомлення про помилку, яке браузер покаже користувачеві (наприклад, «Будь ласка, заповніть це поле» у разі відсутності обов’язкового значення).

validity: повертає об’єкт `ValidityState`, що містить детальні логічні прапорці, які відповідають конкретним випадкам невдалої перевірки.

Підтвердження пароля

Практика: forms

Для реалізації повторного введення пароля (підтвердження пароля) нам потрібно додати нове поле в HTML та розширити логіку JavaScript. Валідація буде перевіряти дві речі: чи заповнене поле та чи збігається його значення з першим паролем.

```
<!-- Додаємо нове поле для підтвердження пароля та блок для виведення помилки. -->
        <!-- Поле пароля (залишається, додаємо лише ідентифікатор) -->
        <div class="form-group">
            <label for="password">Пароль:</label>
            <input type="password" id="password" name="password" required minlength="6">
        </div>
        <!-- НОВЕ: Поле підтвердження пароля -->
        <div class="form-group">
            <label for="confirmPassword">Підтвердіть пароль:</label>
            <input type="password" id="confirmPassword" name="confirmPassword"
placeholder="Повторіть ваш пароль" required>
            <!-- Місце для виведення тексту помилки -->
            <span id="passwordError" class="error-text"></span>
        </div>
```

Логіка валідації

Практика: forms

1. Якщо поле «Підтвердження пароля» порожнє під час відправки — виводиться помилка.
2. Якщо символи хоча б на один знак відрізняються від основного пароля — форма блокується і з'являється повідомлення «Паролі не збігаються».
3. Завдяки живому обробнику подій, як тільки користувач допише правильний символ і паролі стануть однаковими, червона помилка та рамка зникнуть автоматично.

Нам потрібно порівнювати значення поля пароля і поля повторного введення пароля (підтвердження) у реальному часі за допомогою події **input** та блокувати відправку форми, якщо вони не збігаються.

Функція для перевірки збігу

Практика: forms

```
function validatePasswords() {
        const password = passwordInput.value;
        const confirmPassword = confirmPasswordInput.value;
        // Якщо поле підтвердження ще порожнє, не показуємо помилку завчасно
        if (!confirmPassword) {
            passwordError.textContent = '';
            confirmPasswordInput.setCustomValidity('');
            return;
        }
        if (password !== confirmPassword) {
            passwordError.textContent = 'Паролі не збігаються!';
            // Вбудований метод HTML5, який блокує submit форми та показує помилку браузера
            confirmPasswordInput.setCustomValidity('Паролі повинні збігатися.');
        } else {
            passwordError.textContent = '';// Очищаємо помилку, тепер форма валідна
            confirmPasswordInput.setCustomValidity('');
        }
    }
```

Запускаємо перевірку у реальному часі

Практика: forms

```
// Ми додаємо слухач подій input на обидва поля паролів.
// Метод setCustomValidity('') робить поле валідним, а setCustomValidity('текст') — позначає його як помилкове та блокує відправку форми браузером.

    // Отримуємо елементи паролів та блоку помилки

    const passwordInput = document.getElementById('password');
    const confirmPasswordInput = document.getElementById('confirmPassword');
    const passwordError = document.getElementById('passwordError');


// Запускаємо перевірку щоразу, коли користувач щось друкує в будь-якому з полів
    passwordInput.addEventListener('input', validatePasswords);
    confirmPasswordInput.addEventListener('input', validatePasswords);
```

Додатковий CSS для помилок

Практика: forms

```css
.error-text {
 color: #dc3545;
 font-size: 0.85rem;
 margin-top: 5px;
 display: block;
}
/* Опціонально: підсвічування поля червоним, якщо воно невалідне */
.form-group input:invalid:focus {
 border-color: #dc3545;
 outline-color: #dc3545;
}
```

Додати іконку «ока» для показу або приховання пароля

Практика: forms

```
<div class="form-group">
  <label for="password">Пароль:</label>
   <div class="password-wrapper">
    <input type="password" id="password" name="password" required minlength="6">
    <button type="button" class="toggle-password" data-target="password" aria-label="Показати пароль">
    <!-- Іконка відкритого ока (за замовчуванням прихована) -->
    <svg class="eye-open hidden" viewBox="0 0 24 24" width="20" height="20" stroke="currentColor"
      stroke-width="2" fill="none"> <path d="M1 12s4-8 11-8 11 8 11 8-4 8-11 8-11-8-11-8z"></path>
      <circle cx="12" cy="12" r="3"></circle></svg>
      <!-- Іконка закритого ока (показується першою) -->
      <svg class="eye-closed" viewBox="0 0 24 24" width="20" height="20" stroke="currentColor"
       stroke-width="2" fill="none"><path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45
18.45 0 0 1 5.06-5.94M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19m-6.72-
1.07a3 3 0 1 1-4.24-4.24"> </path><line x1="1" y1="1" x2="23" y2="23"></line></svg>
                </button>
              </div>
          </div>
```

Додати іконку «ока» для показу або приховання пароля

Практика: forms

```
<!-- НОВЕ: Поле підтвердження пароля -->
        <div class="form-group">
            <label for="confirmPassword">Підтвердіть пароль:</label>
            <div class="password-wrapper">
                <input type="password" id="confirmPassword" name="confirmPassword" required>
                <button type="button" class="toggle-password" data-target="confirmPassword"
                    aria-label="Показати підтвердження пароля"><svg class="eye-open hidden" viewBox="0
0 24 24" width="20" height="20" stroke="currentColor" stroke-width="2" fill="none"><path d="M1 12s4-8
11-8 11 8 11 8-4 8-11 8-11-8-11-8z"></path><circle cx="12" cy="12" r="3"></circle> </svg>
                    <svg class="eye-closed" viewBox="0 0 24 24" width="20" height="20" stroke="currentColor"
                        stroke-width="2" fill="none"> <path
                            d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-
5.94M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19m-6.72-1.07a3 3 0 1 1-4.24-
4.24"></path> <line x1="1" y1="1" x2="23" y2="23"></line></svg></button>
            </div>
            <!-- Місце для виведення тексту помилки -->
            <span id="passwordError" class="error-text"></span>
        </div>
```

CSS для позиціонування ока

Практика: forms

```
.password-wrapper {
        position: relative;
        display: flex;
        align-items: center;
    }

    .password-wrapper input {
        padding-right: 40px;
        /* Залишаємо місце для іконки справа */
    }
```

CSS для позиціонування ока

Практика: forms

```css
.toggle-password {
    position: absolute;
    right: 10px;
    background: none;
    border: none;
    cursor: pointer;
    font-size: 16px;
    padding: 0;
    line-height: 1;
    user-select: none;
    /* Щоб текст іконки не виділявся випадково */
}

/* Прибираємо стандартний синій контур при кліку на іконку */
.toggle-password:focus {
    outline: none;
}
```

Код для перемикання видимості

Практика: forms

```
document.querySelectorAll('.toggle-password').forEach(button => {
        button.addEventListener('click', function () {
            // Знаходимо потрібний інпут за допомогою id, вказаного в data-target
            const targetId = this.getAttribute('data-target');
            const passwordInput = document.getElementById(targetId);

            if (passwordInput.type === 'password') {
                passwordInput.type = 'text';
                this.textContent = '🙈'; // Змінюємо іконку на заплющені очі
            } else {
                passwordInput.type = 'password';
                this.textContent = '👁️'; // Повертаємо відкриті очі
            }
        });
    });
```

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua